R65 Performance Benchmark

6502 Assembler and R65 Pascal on MacBook and Raspberry Pi 5

This note records benchmark measurements made while examining the historical feasibility of the MILL/TEKMILL artificial intelligence on original R65 hardware. An optimized 6502 Sieve of Eratosthenes and an equivalent R65 Pascal implementation were run on the current R65 emulator on two modern hosts: a MacBook and a Raspberry Pi 5.

1. Historical 6502 Sieve

The benchmark is based on the optimized historical 6502 Sieve of Eratosthenes. It uses an 8192-byte flag array, finds 1899 primes, and was reported at 0.74 seconds on the original 6502 system. The optimized implementation represents odd numbers and stops the composite-clearing work once the relevant prime range has been passed.

2. Conversion to the R65 assembler

The source was adapted to the R65 assembler. Symbols are limited to six letters/numbers, so names such as A.PNTR and GENERATE.PRIMES became APNTR and GENPRM. Apple-specific output and timing code was removed so that the benchmark measures the Sieve computation.

Useful R65 assembler syntax rediscovered during the conversion:

```
LABEL EQU $1234 symbolic constant ORG $2000 program origin LDA =4 immediate constant LDA
=LABEL immediate high byte ASL A explicit accumulator mode ROL A NAME BSS 2 reserve two
bytes NAME WRD value initialized word END
```

Address	Use
$2000-$2080	Sieve program
$3500-$54FF	8192-byte flag array
$5500	End of flag array
$06/$07	APNTR
$08/$09	BPNTR
$1B/$1C	PRIME
$1D/$1E	COUNT
$1F	TIMES

The final source assembled with zero errors and was invoked with GSB 2000. A one-run correctness test returned COUNT = $076B = 1899.

3. Assembler benchmark

A single assembler Sieve is too fast for convenient stopwatch timing. The test therefore uses an inner loop of 255 executions and an outer loop of 40 calls, for a total of **10,200 Sieve executions**.

Host	10,200 runs	One Sieve	Speed vs. 0.74 s 6502
MacBook	70.91 s	6.952 ms	106.4x
Raspberry Pi 5	171.6 s (2:51.6)	16.824 ms	44.0x

The Raspberry Pi 5 therefore runs this CPU-bound emulation benchmark about **2.42 times slower** than the MacBook. Even so, it still emulates the workload at approximately **44 times the speed of the historical 6502**.

> **Measured emulator CPU speed: MacBook approximately 106x original 6502; Raspberry Pi 5 approximately 44x original 6502.**

4. R65 Pascal benchmark

The same Sieve algorithm was implemented in R65 Pascal, whose virtual machine uses 16-bit integers. RUNS=1 and RUNS=10 were measured on each host so that the small program startup/loading component could be separated from the repeated Sieve execution time.

Host	RUNS=1	RUNS=10	Derived one Sieve	Startup
MacBook	0.67 s	6.56 s	0.6544 s	~0.016 s
Raspberry Pi 5	1.66 s	16.38 s	1.6356 s	~0.024 s

For each machine, if S is startup time and T is one Pascal Sieve, then the measurements give S + T for RUNS=1 and S + 10T for RUNS=10. Thus T = (time10 - time1) / 9.

Metric	MacBook	Raspberry Pi 5
Assembler, one Sieve	6.952 ms	16.824 ms
Pascal, one Sieve	0.6544 s	1.6356 s
Pascal / assembler	94.1x	97.2x
Host speed vs. historical 6502	106.4x	44.0x

The most important cross-check is the Pascal/assembler ratio. Although the two hosts differ in absolute speed by roughly 2.4 to 2.5 times, the interpreter/native-code ratio remains almost unchanged: **94.1x on the MacBook and 97.2x on the Raspberry Pi 5**. This independently reproduces the approximately 100x interpreter cost for this particular tight computational workload.

5. Estimated original-speed R65 Pascal

The assembler benchmark supplies an empirical host-to-original-6502 speed ratio. Applying that independently to each host's Pascal result gives an estimate of how long the Pascal Sieve would take on an original-speed R65:

Derived from	Calculation	Estimated original R65 Pascal
MacBook	0.6544 s x 106.4	~69.6 s
Raspberry Pi 5	1.6356 s x 44.0	~72.0 s

> **The two completely separate host measurements therefore predict essentially the same result: about 70 seconds for one R65 Pascal Sieve on original-speed 6502 hardware. Given that the timings were made with a stopwatch on two very different host computers, the agreement is notably good.**

6. Historical comparison

System / implementation	One Sieve
Historical 6502, optimized assembler	~0.74 s

System / implementation	One Sieve
R65 Pascal on current MacBook emulator	~0.654 s
R65 Pascal on Raspberry Pi 5 emulator	~1.636 s
6502 assembler on current MacBook emulator	~0.00695 s
6502 assembler on Raspberry Pi 5 emulator	~0.01682 s

A striking consequence is that interpreted R65 Pascal on the present MacBook executes this benchmark in approximately the same wall-clock time as highly optimized assembler did on the historical 6502. The Raspberry Pi 5, despite being slower than the MacBook in this emulator, still executes the native 6502 benchmark about 44 times faster than the original processor.

7. Consequences for MILL / TEKMILL

These measurements support the conclusion that computationally intensive parts of the MILL AI would have benefited greatly from assembler optimization on original R65 hardware. A historically plausible implementation could keep the overall game, user interface, board handling and higher-level strategy in Pascal while moving the innermost AI search and evaluation loops into 6502 assembler.

Current Mac emulator	Approx. original-speed R65
10 ms	1 s
100 ms	10 s
500 ms	50 s
1 s	100 s

The 100:1 rule is a convenient approximation for CPU-bound work on the MacBook. It does not apply directly to disk I/O, terminal communication or Tektronix drawing.

8. Tektronix and PDP-11 context

The Tektronix side appears much less limiting than the AI computation. TEKMILL experiments showed little visible difference between 100 kbaud and 19,200 baud; 9600 baud remains practical, while 2400 baud produces the sort of redraw delays associated with historical modem-connected graphics terminals.

The PDP-11 is relevant as historical context. Considerable programming was done in C on a PDP-11 using a Tektronix 4010, as well as in interpreted Pascal on the PDP-11, although that work has been lost. An original-speed PDP-11 emulator is still available. Porting the current C R65 emulator to it would require adaptation to the older C grammar and is not necessary merely to establish the present R65 performance comparison.

9. Conclusions

MacBook: 6.95 ms assembler Sieve, 0.654 s Pascal Sieve, about 106x historical 6502 speed, and a 94x Pascal/assembler ratio.
Raspberry Pi 5: 16.82 ms assembler Sieve, 1.636 s Pascal Sieve, about 44x historical 6502 speed, and a 97x Pascal/assembler ratio.

Independent estimate for original R65 Pascal: approximately 70 seconds per Sieve from either host.

The Raspberry Pi 5 result is particularly impressive in historical terms: a small single-board computer can run this emulated 6502 workload at roughly 44 times the speed of the original processor, while simultaneously executing the complete R65 emulator and its surrounding host environment. More importantly for the benchmark, the Pi and Mac results corroborate each other: absolute host performance changes substantially, but the measured R65 Pascal versus optimized assembler ratio remains very close to 100:1.

Benchmark measurements recorded October 2026. Times are wall-clock stopwatch measurements and should be understood as practical engineering benchmarks rather than cycle-exact processor characterization.